Use of AI assistants. Problems 6.1 to 6.4 are No AI. Problem 6.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Haskell file**, built from the template: code as code, traces and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper.

Two rules from the lecture apply to every problem. **No partial functions**: a function without an equation for some shape of its input, or a call to `head`, `tail` or `!!`, loses the working points of its part, whatever its answer. **The file loads**: a part you have not finished stays `undefined`, so that the rest can be marked by running it.

Problem 6.1 (Rides, by hand). No AI A bike-sharing station logs the duration of every ride, in minutes. Write the three functions below **by explicit recursion**, with an equation for each shape of the list and no `map`, `filter`, `sum` or `length`.

(a) `overtime`: the total number of minutes above a limit, over all rides.

```
overtime 30 [12, 45, 8, 61, 30]   -- 46
overtime 60 [12, 45]              -- 0
```

(b) `longRides`: the rides longer than the limit, in their order.

```
longRides 30 [12, 45, 8, 61, 30]  -- [45,61]
longRides 100 [12, 45, 8, 61, 30] -- []
```

(c) `splitAtLimit`: the rides of at most the limit and the rides above it, both in their order, in **one pass** over the list.

```
splitAtLimit 30 [12, 45, 8, 61, 30]  -- ([12,8,30],[45,61])
splitAtLimit 30 []                   -- ([],[])
```

(d) Trace `overtime 30 [45, 8]` by your equations, one step per line, as a comment.

Problem 6.2 (Three ways to choose). No AI Consider `cost`, the price of a ride in rubles: free up to 15 minutes, 50 rubles up to 30 minutes, then 50 and 3 more for every minute above 30, and everything doubled for a ride at night.

```
cost :: Bool -> Int -> Int
cost night minutes
  | minutes <= 15 = 0
  | minutes <= 30 = 50 * factor
  | otherwise     = (50 + 3 * (minutes - 30)) * factor
  where
    factor = if night then 2 else 1
```

```
cost False 10   -- 0
cost False 40   -- 80
cost True 40    -- 160
```

(a) Rewrite `cost` as a **`case`** on the pair `(night, minutes)`, with tuple patterns and guards inside the alternatives.

(b) Rewrite it with `if-then-else` only, with no guards and no `case`. Then say in one sentence what the `where` still saves.

(c) The `Bool` argument shows one of the three flaws of the lecture's slide *Types That Say Too Little*. Name it, and say in one sentence what a better type would say in its place. No definition is asked for.

Problem 6.3 (Types by hand). No AI Consider four definitions over the given `rides`, `stations` and `docks`, written without signatures:

```haskell
rides    = [12, 45, 8, 61, 30]          -- rides :: [Int]
stations = [Station "Park" 12, Station "Campus" 20, Station "Market" 8]
docks (Station _ d) = d                 -- docks :: Station -> Int

busy    = length rides > 3
perDock s = fromIntegral (docks s) / 2
labels  = [n ++ "!" | Station n _ <- stations]
total   = sum rides + length rides
```

(a) Write the type signature of each of the four, and for each a line saying how the type of the body decides it.

(b) Two more expressions: `sum rides / 2` and `length rides + 1`. GHC refuses one of them. Say which, and give the *what* part of its message in the lecture's words: the type expected and the type found, or the constraint that cannot be satisfied.

Problem 6.4 (Minutes are not rubles). No AI Consider the module below, given in the template. A refund for a ride cut short is the fare of the planned ride, less the fare of the minutes actually ridden. The example shows a wrong number: `refund` passes a fare where a duration was meant.

```haskell
type Minutes = Int
type Rubles  = Int

fare :: Minutes -> Rubles
fare m
  | m <= 30   = 50
  | otherwise = 50 + 3 * (m - 30)

refund :: Minutes -> Minutes -> Rubles
refund planned ridden = fare planned - fare (fare ridden)
```

```haskell
refund 60 20   -- 30, and 90 was meant
```

(a) Replace the two aliases by `newtype`s, and update `fare` and `refund` so that the module compiles again with the mistake still in it. Paste GHC's message for the mistaken line as a comment. Then fix the mistake.

(b) Consider a ride at a named station, and two functions between it and a pair:

```haskell
data Ride = Ride String Minutes

toPair   :: Ride -> (String, Int)
fromPair :: (String, Int) -> Ride
```

Write the two, and argue in one sentence each that `fromPair (toPair r)` is `r` for every ride and `toPair (fromPair p)` is `p` for every pair.

Problem 6.5 (The busiest station, audited). Consider a list of the stations of the city, each with the number of rides that started there today:

```haskell
counts :: [(String, Int)]
counts = [("Park", 14), ("Campus", 27), ("Market", 9)]
```

`busiest` is the name of the station with the most rides.

(a) AI allowed Ask an assistant to write it, and record what it produces as `busiestAi`. Any follow-up you put to it belongs here too, and in the transcript. A prompt along these lines:

> Write a Haskell function `busiest :: [(String, Int)] -> String` that returns the name of the station with the most rides.

Use a **basic, free** model, not the strongest you can reach.

(b) No AI Mark, in a comment next to the recorded answer, every function that the lecture did not introduce, and every partial function, that is, one that has no answer for some input. Then write `busiest` with lecture 6 only: patterns, and a helper with an accumulator.

(c) No AI Run both on `counts`, on `[("Park", 14)]` and on `[]` in GHCi, and report what each does on the empty list. The lecture named two ways a type could say more here, an optional result or an input that is never empty. Say in one sentence which of the two fits `busiest`, and why.

Use of AI assistants. Part (a) only. If you use an assistant, submit `transcript.md` alongside your answer file: the whole exchange, your prompts and the assistant's replies, verbatim and unedited. Record its function in part (a), but do **not** assume that it is correct. Parts (b) and (c) are answered in the answer file itself, in your own words. An answer to (b) with nothing marked scores nothing. If the recorded function does not load, keep it verbatim in a comment, note the error in (b), and leave `busiestAi` as `undefined`, so that the rest of the file still loads. If you use no assistant, write the function yourself in (a) and audit your own attempt.